

ASSIGNMENT-2 Artificial Intelligence

Manvendra pratap Singh (G24AIT2163)

INDIAN INSTITUTE OF TECHNOLOGY-JODHPUR

Instructor - Dr. Debasis Das

Date 23-April-2025

Question 1

Design a Sudoku puzzle where the board consists of 81 squares, some of which are initially filled with digits from 1 to 9. The puzzle is to fill in all the remaining squares such that no digit appears twice in any row, column, or 3×3 box. A row, column, or box is called a unit.

1. Represent the Sudoku problem in CSP by identifying the variable, domain, and constraint.
2. Implement the problem using backtracking search, what is avg. time taken by the algorithm for 10 runs.
3. Analyse how different fault finding algorithms such as Forward Checking, Arc consistency improve the computational time of backtracking search?
4. Analyse how different Heuristics MRV (Minimum Remaining Values), Degree heuristic, Least Constraining Value affect the computational time of backtracking search?

Note: Prepare Comparison table in your report and provide the reasons for performance improvements.

Solution 1

Sudoku Puzzle Solver using CSP Techniques

Here is the dynamic link of my solution: <https://sudoku-app-manvendrapratapsinghdev.streamlit.app/>

Problem Statement

The goal is to solve a 9×9 Sudoku puzzle using Constraint Satisfaction Problem (CSP) methods. Each empty cell must be filled such that every row, column, and 3×3 subgrid contains digits 1 through 9 without repetition. The problem is modeled in CSP by:

- **Variables:** Each of the 81 cells
- **Domain:** {1, 2, ..., 9} for each empty cell
- **Constraints:** All units (rows, columns, boxes) must contain unique values

Approach & Methodology

We implemented the solver using Python and CSP techniques with three main strategies:

1. **Backtracking Search** – A brute-force search that assigns values recursively, backtracking upon constraint violations.
2. **Forward Checking** – Improves backtracking by eliminating inconsistent values from domains of unassigned variables after each assignment.
3. **Arc Consistency (AC-3)** – Applies constraint propagation by maintaining consistent value pairs between variables.

Heuristics Used

To further optimize solving time, the following heuristics were applied:

- **MRV (Minimum Remaining Values):** Selects variables with the fewest legal values.
- **Degree Heuristic:** Prefers variables involved in more constraints.
- **LCV (Least Constraining Value):** Picks values that impose the fewest constraints on peers.

Implementation Summary

The code initializes the board using a string input and constructs mappings for units and peers. Solving functions are modular: `backtracking_search()`, `forward_checking()`, `arc_consistency_ac3()`, and `apply_heuristics()`. Performance metrics like execution time, number of assignments, backtracks, and nodes explored are recorded and compared via the `compare()` function.

Performance Analysis

Ten runs were conducted for each technique. Average runtime and improvements were observed as follows:

Performance Metrics					
	Method	Time (s)	Assignments	Backtracks	Nodes Explored
0	Backtracking Search	0.0434	4208	23	44
1	Forward Checking	0.0360	4208	7	7
2	Arc Consistency (AC-3)	0.0540	0	88	889
3	Heuristics	0.0125	0	63	114

Observations

- **Forward Checking** significantly reduces the need for backtracking by pruning domains early.
- **Arc Consistency** makes the system more efficient by enforcing consistency ahead of time.
- **Heuristics**, especially MRV and LCV, guide the search more intelligently and minimize failed paths.

Why These Techniques Improve Performance

Each CSP technique adds a layer of **intelligence and efficiency** to the solving process:

1. Forward Checking

- Cuts off invalid paths **early** by removing illegal values from future cells.
- Saves computation by **not pursuing paths** that are already impossible.

2. Arc Consistency (AC-3)

- Goes a step further by **preemptively eliminating value conflicts** across the board.
- Reduces the problem size even **before** the actual search begins.
- This can lead to solving parts of the puzzle without any guessing at all.

3. Heuristics

- These are the **decision-makers**.
- They ensure we don't waste time on "easy" or "non-critical" cells.
- By focusing on the **most constrained variables (MRV)** and **least destructive choices (LCV)**, the solver avoids early mistakes that would require heavy backtracking.

4. Overall Impact

- Fewer variables to check, fewer values to try, and smarter choices.

- Together, these strategies **reduce the number of assignments, backtracks, and total runtime** significantly compared to vanilla backtracking.

Conclusion

This problem successfully demonstrated the application of CSP strategies to Sudoku solving. Enhancements like constraint propagation and heuristics showed measurable performance gains over plain backtracking.

Question 2

Consider a simple N-Queens problem. Take a suitable value of N and write a Python code for simulated annealing to solve the problem. As we know, if we take $T=0$ in SA, it falls back to simple hill climbing search. Modify the SA algorithm in order to make it an HC solution. In the report, write proper theoretical justification in support of your modification (e.g., calculation of the acceptance probability) and the behavioral difference between the two approaches. (Note: No marks will be given for simply implementing the HC solution.)

Solution 2

N-Queens Problem: A Comparative Study Using Simulated Annealing and Hill Climbing

Problem Overview

The N-Queens puzzle requires placing N queens on an $N \times N$ chessboard such that no two queens threaten each other (i.e., no two are in the same row, column, or diagonal). This project focuses on solving the 8-Queens variant ($N=8$) and compares two local search methods—Simulated Annealing (SA) and Hill Climbing (HC)—for efficiency and effectiveness in reaching valid solutions.

Methodology

Two algorithms were implemented:

- **Simulated Annealing (SA):** A probabilistic approach that accepts worse solutions temporarily to avoid local optima, governed by a cooling temperature schedule.
- **Hill Climbing (HC):** A greedy local search method that moves to the best neighbor until no further improvement is possible. To counter stagnation in local optima, it uses random restarts.

Key Concepts Used:

- **Cost Function:** Measures number of queen conflicts.
- **Neighbor Generation:** Moves a queen within its column to new rows.
- **Acceptance Probability (SA only):** Accepts worse states based on probability.
- **Random Restarts (HC only):** Re-initializes the board to escape local optima.

Both algorithms were encapsulated in Python classes handling initialization, neighbor evaluation, and solution search. SA includes cooling logic, while HC emphasizes repeated trials.

Implementation Summary

- **SA Flow:** Start with a random board → generate neighbors → evaluate and accept/reject based on probability → reduce temperature → stop at zero conflicts or max iterations.
- **HC Flow:** Start with a random board → find best neighbor → update if improved → else restart → stop at solution or after defined restarts.

Results & Evaluation

Both algorithms were tested for multiple runs on the 8-Queens setup. Key findings:

Algorithm	Solution Found	Avg. Iterations	Remarks
Simulated Annealing	Yes (most runs)	Moderate	Escapes local optima well
Hill Climbing	Yes (after restarts)	Fewer per trial	Fast but often stuck locally

- **Simulated Annealing** proved more robust due to its exploration capability.

- **Hill Climbing** was faster per run but required multiple restarts for success.

Conclusion

SA and HC both effectively solve the N-Queens problem, with trade-offs. SA is ideal for escaping local optima, whereas HC can be faster if the board is favorable. Future enhancements could include adaptive parameter tuning or hybrid models combining both strategies.
